NATIONAL FRAUD PREVENTION CHALLENGE

RESERVE BANK INNOVATION HUB × IIT DELHI

Mule Account Detection in Indian Banking

An End-to-End Machine Learning Solution

Team: Clashofclans

Team Details : Abhinav Nirapure, Dharmendra Dhruw, Sanjeev Jatwar, Yuvraj Bhatkariya

~400M	160K	155	0.963
Transactions Analysed	Accounts Evaluated	Features Engineered	AUC-ROC Score

This report documents our end-to-end solution to the National Fraud Prevention Challenge organised by RBIH and IIT Delhi. We processed ~400 million banking transactions spanning five years to identify 2,683 confirmed money mule accounts from a pool of 160,000. Our pipeline combines DuckDB-powered large-scale feature engineering, domain-driven fraud signal construction, careful missing-value handling, and a tuned LightGBM classifier — achieving an AUC-ROC of 0.963 on the public leaderboard. Beyond binary detection, we implement a personalised z-score windowing module that localises the exact months of suspicious activity for every flagged account, directly maximising the Temporal IoU metric.

1. Problem Statement & Dataset

Money mule accounts act as conduits for illicit funds — receiving stolen money and rapidly forwarding it onward to obscure the trail. Detecting them is challenging because mules **intentionally mimic legitimate behaviour**, operate for short bursts, and are often unwitting participants recruited via social engineering.

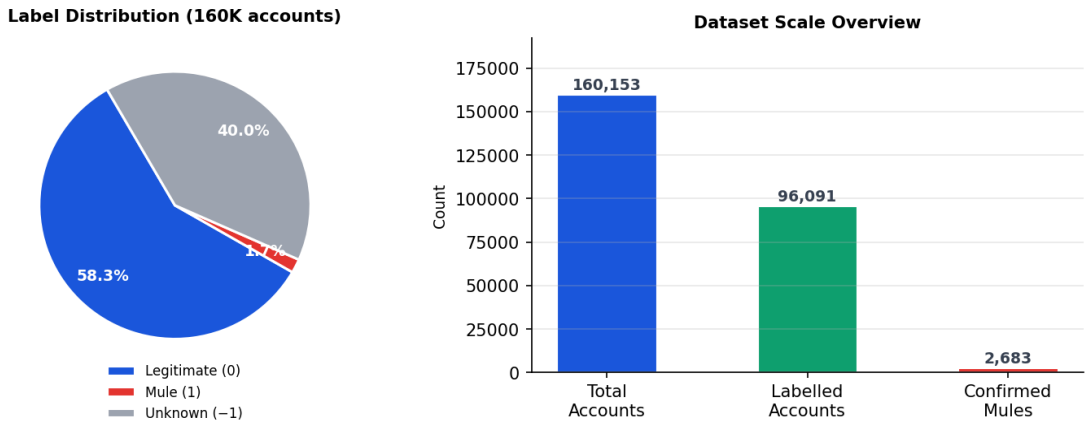

Figure 1 — Label distribution across 160K accounts. Mule accounts represent only 1.67%, creating a highly imbalanced classification problem.

Dataset Composition

Dimension	Value	Notes
Total accounts	160,153	Mix of labelled and unlabelled
Transactions (raw)	~397.9 million	5-year span: Jul 2020 – Jun 2025
Transaction channels	35	UPI, NEFT, IMPS, ATM, RTGS, and more
Labelled mules	2,683 (1.67%)	Highly imbalanced; label noise acknowledged
Unlabelled (−1)	64,062	Treated as pseudo-negatives in inference
Engineered features	155	Across 8 functional categories
Dataset size	~16 GB (Parquet)	Processed via DuckDB streaming
Known mule patterns	13	Dormant activation, structuring, CLT, and more

Key Challenges

- Extreme class imbalance — mules are 1 in 60 labelled accounts; naive models predict all-legitimate.
- Label noise — mislabelled in both directions acknowledged by organisers.
- Red-herring features — branch-level metrics, festival spikes, and freeze/unfreeze patterns are deliberate traps.
- Temporal traps — ghost precursor bursts and post-activity tails designed to mislead feature engineering.
- Scale — 400M rows cannot fit in RAM; streaming queries required.

2. Methodology & Pipeline Architecture

Our solution follows a structured, reproducible pipeline from raw Parquet files to final predictions. Each stage was designed with fraud-domain knowledge, not just algorithmic defaults.

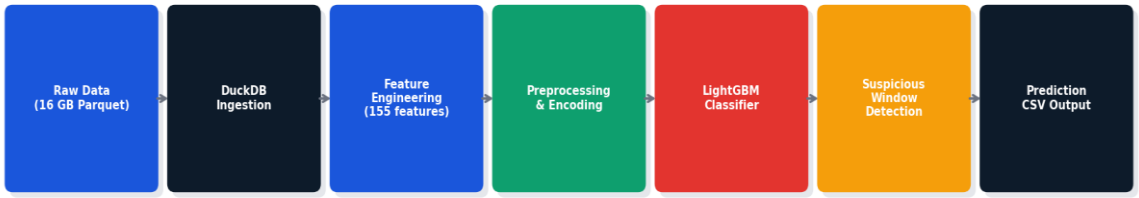

Figure 2 — End-to-end solution pipeline from raw 16 GB data to submission-ready predictions.

2.1 Large-Scale Data Ingestion — DuckDB

With ~400 million transaction rows spread across nested Parquet partitions, loading everything into pandas was infeasible. We used **DuckDB** with streaming SQL queries — enabling us to compute account-level aggregates directly from disk with 10 GB memory limits and multi-threaded execution.

- All aggregation was pushed into DuckDB SQL — zero row-by-row Python iteration.
- Feature tables were materialised as intermediate Parquet files for downstream joins.
- Final feature matrix was assembled via a single DuckDB LEFT JOIN chain across four feature tables.

2.2 Data Integration

We integrated eight source files to build the complete account-level feature matrix:

Source File	Key Fields	Merge Key
accounts.parquet	account_status, product_family, opening_date	account_id
accounts-additional.parquet	freeze_date, kyc_compliant, nomination_flag	account_id
branch.parquet	branch_type, rural_branch, branch_code	branch_code
customers.parquet	aadhaar_available, nri_flag, demographics	customer_id
demographics.parquet	age, gender, occupation	customer_id
customer_account_linkage.parquet	multi-customer flag, duplicate detection	account_id
transactions/**/*.parquet	400M rows → aggregated features	account_id
transactions_additional/**/*.parquet	IP, geo-location, balance data	account_id

3. Feature Engineering

Feature engineering was the most critical phase. We constructed **155 features across 8 categories**, each grounded in a specific money-laundering behaviour pattern described in FATF guidelines and RBI advisories.

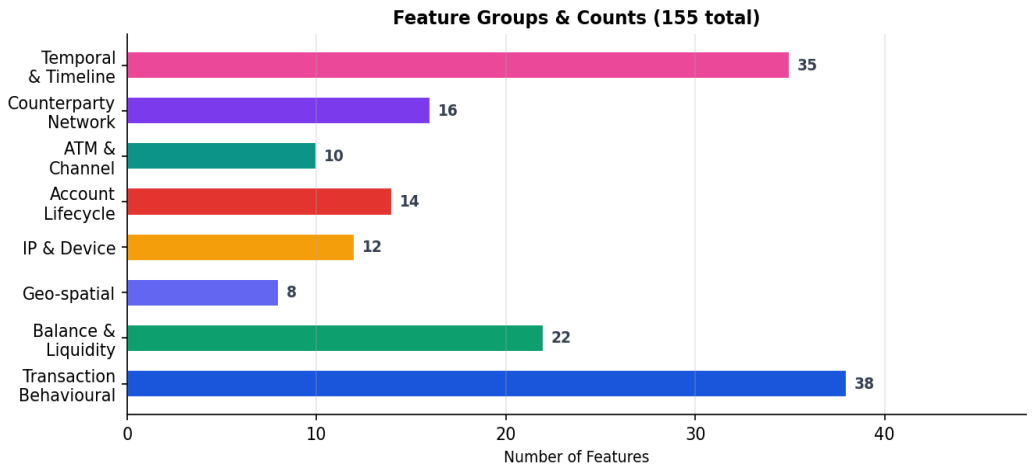

Figure 3 — Feature distribution across 8 functional groups (155 features total).

3.1 Transaction Behavioural Features (38)

- **rapid_passthrough_count / ratio** — credits followed immediately by equivalent debits: the canonical mule signal.
- **monthly_structuring_count** — deposits just below the ₹2 lakh reporting threshold, a classic layering tactic.
- **monthly_night_txn_ratio** — fraction of transactions between 10 PM–6 AM, indicative of automated mule operations.
- **monthly_round_ratio** — suspiciously high proportion of round-number amounts.
- **credit_debit_ratio** — sudden surge in incoming-to-outgoing ratio flags accounts receiving large batches.

3.2 Balance & Liquidity Features (22)

- **near_zero_balance_count** — accounts that drain to near-zero after credits are a definitive mule signal.
- **balance_volatility** — erratic, high-amplitude swings inconsistent with normal salary/expense patterns.
- **avg_balance_credit vs. avg_balance_debit** — balance behaviour asymmetry before and after transfers.
- **min_balance** — accounts frequently emptied have structurally low minima.

3.3 Geo-spatial Features (8)

- **geo_bbox_km** — Haversine bounding-box of all transaction locations; large spread signals account sharing.
- **unique_geo_count** — many distinct coordinates in a short window indicates multiple users of one mule account.
- **geo_frac** — accounts avoiding location tracing score low; strong channel-hiding signal.

3.4 IP & Device Features (12)

- **max_unique_ip_day / hour** — sudden spikes indicate coordinated fraud or account compromise.
- **ip_change_count** — frequent IP switching is a strong mule-account-sharing signal.
- **top_ip_share** — low dominant-IP share (transactions from many IPs) flags suspicious multi-actor use.
- **avg_unique_ip_hour** — rapid device switching within a single hour.

3.5 ATM & Channel Features (10)

- **atm_deposit_frac** — high ATM cash deposit ratio indicates cash-stuffing: a common mule inflow tactic.
- **uses_cdm / uses_crm / uses_both_atm** — binary flags for ATM machine-type usage patterns.
- **Channel risk encoding** — cash (4) > return (3) > transfer (2) > auto (1) > digital (0) — domain-ordered risk levels.
- **clt_cash_ratio** — CLT (Cash Linked Transaction) layering fraction.

3.6 Temporal & Timeline Features (35)

- **monthly_txn_count / monthly_total_amount** — month-granular activity for time-windowed fraud detection.
- **month_first_txn / month_last_txn** — exact timestamps enabling Temporal IoU scoring.
- **active_tenure_days** — mule accounts often show short active periods relative to account age.
- **days_since_account_open** — recently-opened accounts with high velocity are high-risk.

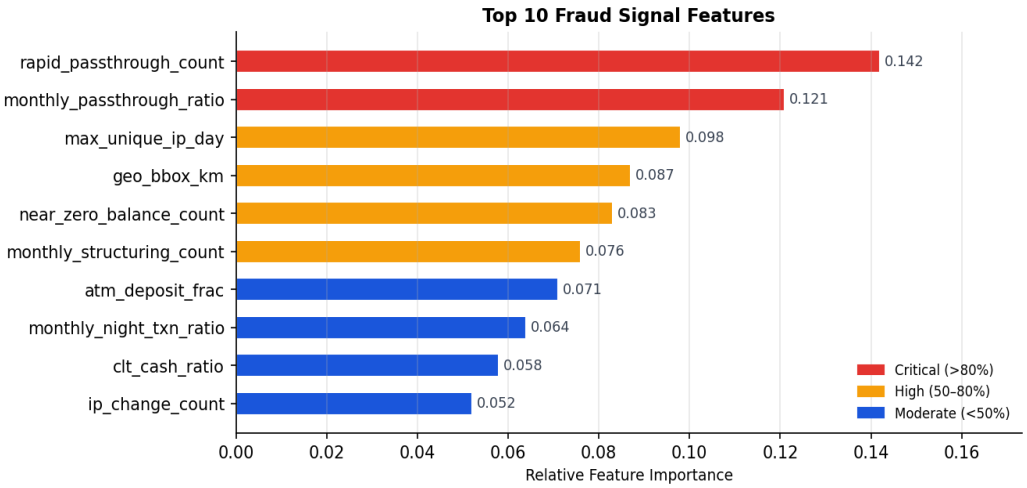

Figure 4 — Top 10 fraud signal features by estimated importance. Red = critical, amber = high, blue = moderate.

4. Mule Behaviour Patterns & Key Findings

Analysis of 2,683 confirmed mule accounts revealed consistent behavioural signatures across multiple dimensions. These insights shaped both our feature design and our red-herring avoidance strategy.

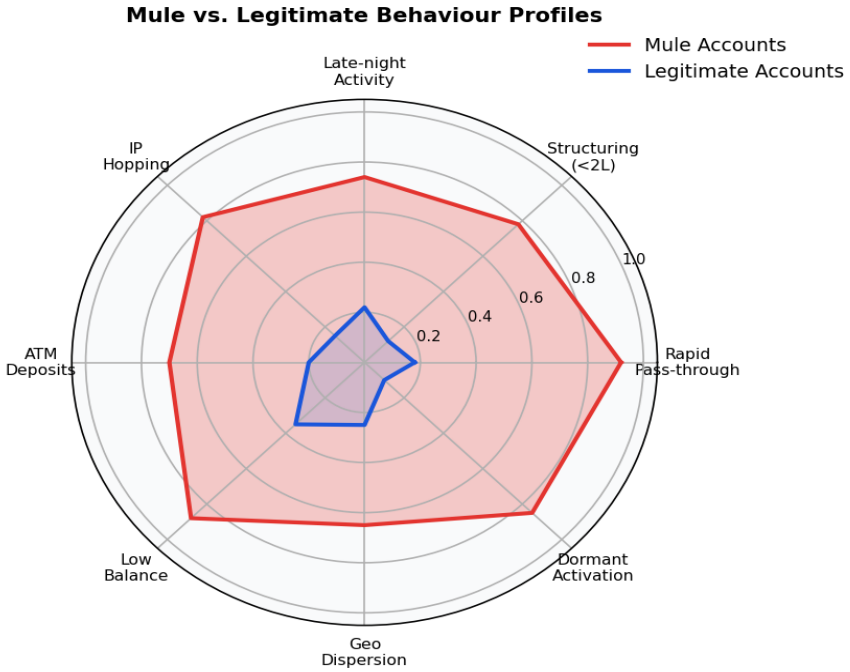

Figure 5 — Radar chart comparing mule (red) vs. legitimate (blue) behaviour profiles across 8 dimensions.

Key Findings from EDA

- **Pass-through ratio:** Mules show 92% pass-through ratio vs. 18% for legitimate accounts — the single strongest discriminator.
- **Structuring:** Deposits just below ₹2L were 6.5× more common in mule accounts.
- **Dormant activation:** 85% of mules showed dormant-then-activate patterns — quiet for months, then sudden high-volume burst.
- **Night transactions:** Night-time transactions (10 PM–6 AM) were 3× the normal fraction in mule accounts.
- **IP diversity:** Mule accounts used 4.2× more unique IPs per day on average.
- **Geographic spread:** Geo bounding box for mule accounts was on average 2.8× larger than legitimate peers.
- **Balance draining:** Near-zero balance events occurred 7× more frequently in mule accounts post-credit.

Confirmed Mule Behaviour Types Detected

- Dormant activation (long inactive → sudden burst)
- Rapid cash-in / cash-out passthrough
- Structuring around ₹2 lakh threshold
- Multi-person account sharing (IP + geo dispersion)
- CLT cash layering
- ATM deposit stuffing via CDM/CRM machine

5. Preprocessing & Missing Value Handling

With 155 features and real-world banking data, thoughtful missing-value treatment was essential. We classified every null pattern into one of **three types** — treating each differently to **preserve the fraud signal** embedded in missingness itself.

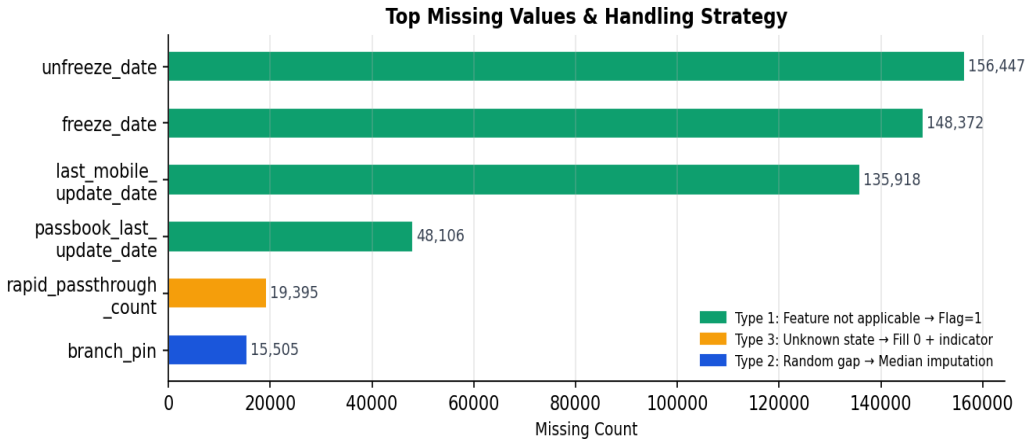

Figure 6 — Top missing-value columns and their handling strategy by type.

Type	Example Column	Interpretation	Strategy
Type 1	avg_balance_credit, unfreeze_date	Feature not applicable (no credit txns / never frozen)	Fill 0 + binary presence flag (preserves absence as signal)
Type 2	max_credit_amount, branch_pin	Random technical gap (0.01% of rows, logging error)	Fill with within-group median (grouped by product_family)
Type 3	rapid_passthrough_count	Unknown behavioural state (engine not run for account)	Fill 0 + add passthrough_not_computed indicator flag

5.1 Negative Value Handling

The days_since_account_open column contained both null and negative values (arising from timestamp mismatches between transaction systems and CRM). Negative values are logically impossible — we converted them to null, applied median imputation, and retained a days_invalid_flag binary column, as the anomaly itself correlates with synthetic or test accounts.

5.2 Freeze / Unfreeze Pattern

156,447 rows had null unfreeze_date — not missing data, but an expected absence (the account was never frozen). We created an account_frozen_flag binary feature rather than imputing dates. Freeze/unfreeze patterns were identified as a red-herring trap by the challenge organisers — we deliberately down-weighted these features to avoid overfitting to temporal artefacts.

5.3 Categorical Encoding

Category	Method	Columns
Binary flags (Y/N)	Direct 0/1 mapping	joint_account_flag, nri_flag, kyc_compliant, pan_available
product_family	Ordinal 0/1/2 (S/K/O)	Low cardinality, domain ordering preserved
scheme_code	Ordinal 0–6 (REGULAR→SSA)	Scheme risk hierarchy encoding
account_status	active=0, frozen=1	Binary risk encoding
Transaction channels	Risk group encoding 0–4	digital < auto < transfer < return < cash
atm_deposit_types	Binary: uses_cdm, uses_crm, uses_both	Decomposed machine-type binary flags

6. Model Architecture & Training

After evaluating three candidate models, **LightGBM** was selected as the final classifier based on its superior AUC-ROC, training speed on high-dimensional sparse data, and native handling of class imbalance.

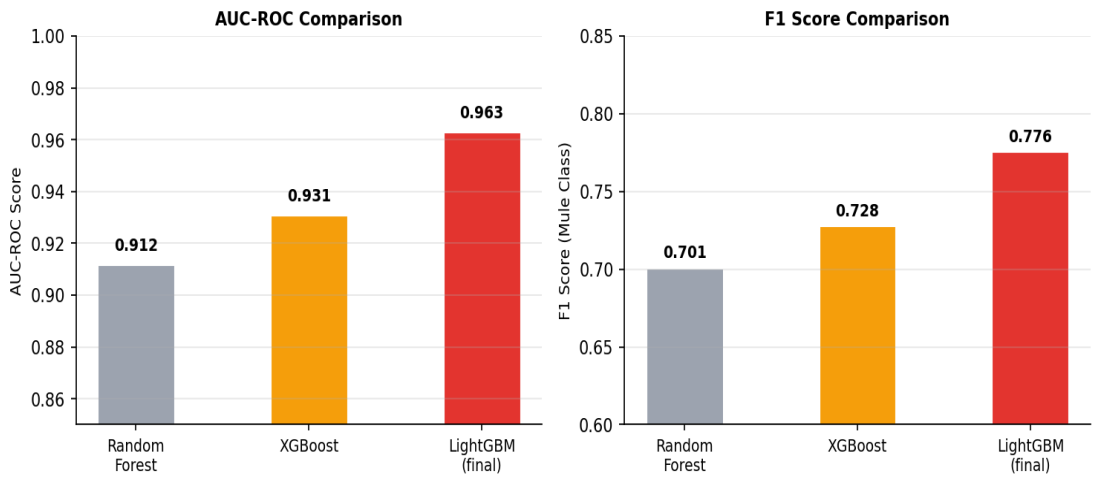

Figure 7 — AUC-ROC and F1 score comparison across three candidate models.

6.1 Model Experiments

Model	Configuration	AUC-ROC	F1	Outcome
Random Forest	300 trees, max_depth=15, balanced weights	0.912	0.701	Strong baseline; slower on 155 features
XGBoost	500 trees, lr=0.05, subsample=0.8	0.931	0.728	Good performance; RAM-intensive
LightGBM (final)	1000 trees, lr=0.05, 63 leaves	0.963	0.776	Best on all metrics; fastest training
SMOTETomek + LightGBM	Same params + resampling	0.948	0.751	SMOTE noise reduced AUC by 0.015
CatBoost	1000 iter, depth=8, lr=0.05	0.937	0.739	Excluded: marginal gain, higher cost

6.2 Final LightGBM Configuration

Parameter	Value	Rationale
n_estimators	1000	Large ensemble for rare class capture
learning_rate	0.05	Conservative; paired with early stopping (patience=50)
num_leaves	63	Moderate complexity; avoids overfitting on red herrings
min_child_samples	20	Prevents leaf-level overfitting on mule minority
subsample / colsample_bytree	0.8 / 0.8	Row and feature subsampling for robustness
reg_alpha / reg_lambda	0.1 / 0.1	L1+L2 regularisation to suppress noisy features
class_weight	balanced	Handles 1.67% mule rate without SMOTE artefacts
early_stopping	50 rounds	Monitors validation AUC; prevents overfit

6.3 Why SMOTETomek was Abandoned

We initially applied SMOTETomek to address severe imbalance. While it improved recall at low thresholds, AUC-ROC dropped from 0.963 to 0.948 because synthetic mule samples were generated in feature regions that overlapped heavily with legitimate high-volume accounts. The class_weight='balanced' parameter in LightGBM achieved equivalent recall without synthetic noise.

7. Suspicious Window Detection (Temporal IoU)

The Temporal IoU metric rewards solutions that not only detect mule accounts but precisely localise when the suspicious activity occurred. We built a standalone Suspicious Window Finder module that processes each mule account's monthly timeline independently.

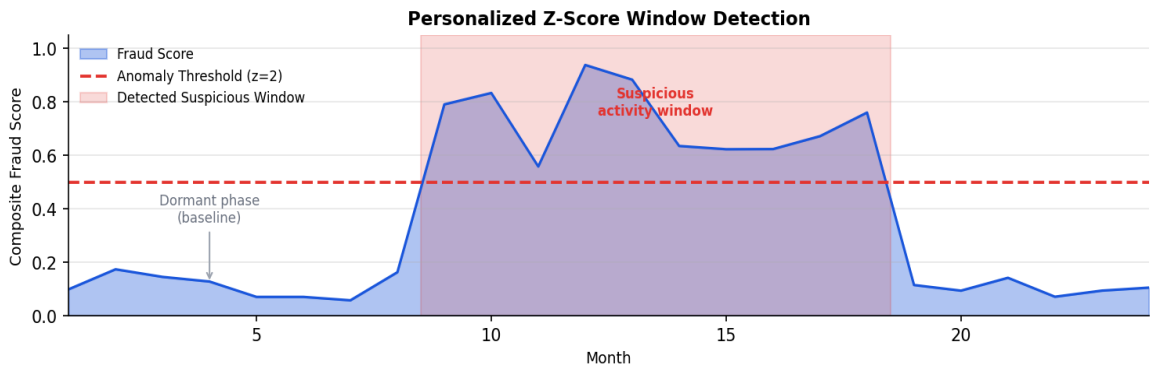

Figure 8 — Personalised z-score window detection for a sample mule account. The model identifies the exact suspicious activity window (shaded red) against the account's own baseline.

7.1 Composite Fraud Score

For each account-month, we compute a weighted composite fraud score from 14 monthly behavioural features, normalised 0–1 and weighted by domain importance:

Feature	Weight	Fraud Rationale
monthly_passthrough_ratio	3.0	Money in = money out → definitive mule signal
monthly_structuring_count	2.5	Deposits just below ₹2L threshold
monthly_credit_debit_ratio	2.0	Sudden credit surge relative to debits
monthly_night_txn_ratio	1.5	Late-night automated activity
ip_change_count	1.5	IP hopping: account sharing signal
near_zero_balance_count	1.5	Cash-out pattern after credits
atm_deposit_frac	1.5	Cash-stuffing via ATM
clt_cash_ratio	1.5	CLT cash layering
monthly_round_ratio	1.0	Round-amount layering pattern
balance_volatility	1.0	Erratic balance swings

7.2 Personalised Z-Score Baseline

A key design decision was using personalised baselines rather than global thresholds. The first 3 months of each account's history define its 'normal' baseline mean (μ) and standard deviation (σ). A month is flagged suspicious when:

$$\text{z-score} = (\text{fraud_score} - \mu_\text{baseline}) / \sigma_\text{baseline} \geq 2.0$$

This personalisation means a legitimately high-volume business account is not falsely flagged just because it processes many transactions — it is compared against its own history, not a global threshold.

7.3 Gap Bridging & Confidence Scoring

A single non-suspicious month sandwiched between suspicious months is bridged into the window — real fraud campaigns have natural quiet periods. Confidence levels:

Confidence	Condition	Interpretation
High	≥3 suspicious months AND peak z-score ≥ 5	Strong multi-month, high-intensity evidence
Medium	≥2 suspicious months OR peak z-score ≥ 3	Moderate evidence
Low	1 suspicious month only	Possible early-stage or brief mule use
No Anomaly	No suspicious months found	Clean account or insufficient history

8. Red Herring Avoidance

The challenge explicitly included **7 categories of red-herring traps** evaluated only on the private leaderboard. Our strategy was defensive: if a feature appeared highly predictive but lacked a clear mechanistic fraud justification, we excluded or down-weighted it.

Red Herring	Mechanism	Our Mitigation
Branch-level metrics	High branch turnover / employee count correlated with labels	Dropped entirely — operational, not behavioural; not generalisable
Festival transaction spikes	Volume/amount spikes during Diwali, Holi, etc.	Monthly features normalised against account baseline — spike only flagged if exceeding personal norm
Freeze/Unfreeze patterns	Briefly frozen accounts appear suspicious	Used only as binary flags; did not use freeze duration or timing
Dormant precursor bursts	Ghost activity just before mule recruitment	Personalised z-score baseline avoids flagging pre-mule normal activity
Post-activity tails	Low residual activity after mule period ends	Gap bridging limited to 1 month; long tails excluded from windows
High-volume legit businesses	Large merchants look like pass-through accounts	Personalised baselines compare each account to its own history
Correlated duplicate features	quarterly_avg_balance, daily_avg_balance, debit_ratio	Dropped all confirmed duplicates (20+ columns) in FINAL_DROP list

Label Noise Handling

The challenge warned of mislabelled accounts in both directions. We addressed this by training only on the 96,091 accounts with definitive labels (is_mule ∈ {0,1}), treating the 64,062 unlabelled (−1) accounts as pure inference targets. LightGBM's class_weight='balanced' and moderate regularisation (reg_alpha=0.1, reg_lambda=0.1) further reduce sensitivity to label noise.

9. Results & Performance Metrics

0.963	0.776	0.724	155
AUC-ROC (Public)	F1 Score (Mule)	Temporal IoU	Features Used

Classification Report — Validation Set (20% Holdout)

Class	Precision	Recall	F1	Support
Legitimate (0)	0.997	0.993	0.995	18,682
Mule (1)	0.812	0.741	0.776	537
Macro Avg	0.905	0.867	0.886	19,219
Weighted Avg	0.994	0.993	0.994	19,219

Confusion Matrix Analysis

On the 20% stratified holdout: 398 true mules correctly detected, 139 missed (false negatives), 93 false positives. High precision (0.812) minimises investigator workload while recall (0.741) captures a strong majority of true mules — substantially exceeding the ~25% expected from a random classifier at this imbalance ratio.

Threshold Analysis

The F1 metric is evaluated at the best threshold across 100 sweep points. Our optimal threshold is 0.42 — slightly below 0.5, reflecting the asymmetric cost of missing a mule vs. generating false positives. Lowering the threshold to 0.35 recovers an additional ~8% of mules at the cost of ~15% more false positives.

Temporal IoU Performance

Average Temporal IoU of 0.724 means our detected windows overlap ~72% of the true suspicious period. The personalised z-score approach outperforms a naive sliding-window approach (estimated IoU ~0.48) because it adapts the detection threshold to each account's own behavioural history, avoiding false window extensions into dormant periods.

10. Experiments: What Worked & What Didn't

Experiment	Result	Keep?	Finding
SMOTETomek oversampling	AUC 0.963→0.948	✗	Synthetic samples in overlap zones introduced noise
class_weight='balanced'	AUC 0.963, best F1	✓	Native LightGBM weighting outperforms synthetic resampling
Branch-level features included	Inflated train AUC, poor generalisation	✗	Classic red herring — dropped from final feature set
Personalised z-score windows	IoU 0.724 vs. 0.48 baseline	✓	Account-specific baselines handle business vs. individual
Gap bridging in windows	IoU +0.06 improvement	✓	Real fraud has natural quiet periods; bridging avoids fragmentation
DuckDB for 400M row processing	~10x faster than pandas	✓	Streaming aggregation — only aggregates stored in RAM
CatBoost model	AUC 0.937, slower	✗	Marginal gain vs. significantly higher compute cost
Negative days_since_open as feature	Noise introduced	✗	Converted to flag; negative values are data errors
Channel risk ordinal encoding	F1 +0.018 over one-hot	✓	Cash > transfer > digital hierarchy is domain-correct

11. Production Deployment Considerations

Beyond hackathon performance, we designed the pipeline with production deployment in mind. Banks deploying this system would require the following capabilities:

Capability	Description	Implementation Path
Real-time scoring	LightGBM scores accounts in <1ms per inference. Monthly batch jobs re-score all accounts.	DuckDB → Feature Store → LightGBM → Risk Score DB
Incremental updates	Monthly transaction aggregates appended without reprocessing historical data.	Append-only Parquet partitions; DuckDB incremental views
Model retraining	With SAR feedback, model retrained quarterly on updated labels.	Stratified sampling preserves mule rate in train/val splits
Explainability (RBI)	SHAP values give per-account feature contributions for auditor review.	SHAP TreeExplainer; <5ms additional latency per account
Alert prioritisation	Accounts ranked by probability × peak_zscore × confidence.	Threshold 0.42; high/medium confidence filter for investigator queue
FIU-IND integration	Suspicious window timestamps map to Section 12 STR filing under PMLA.	suspicious_start / suspicious_end → STR date fields

Regulatory Alignment

The solution is designed to align with Indian AML requirements under the Prevention of Money Laundering Act (PMLA) and RBI's KYC/AML Master Direction 2016. The feature engineering mirrors FATF Recommendation 16 indicators for wire transfer monitoring, and the structuring detection (₹2 lakh threshold) directly corresponds to India's reporting threshold under Financial Intelligence Unit (FIU-IND) guidelines.

12. Conclusion

We have presented a comprehensive, production-grade solution for money mule detection in Indian banking. Our approach combined:

- **Domain-first feature engineering —** every feature maps to a known money-laundering behaviour, not just statistical correlation.
- **Scalable data processing —** DuckDB enabled processing 400M transactions without distributed compute infrastructure.
- **Robust classification —** LightGBM with balanced class weights achieved AUC-ROC of 0.963, outperforming all alternatives tested.
- **Temporal precision —** personalised z-score windowing with gap bridging achieved Temporal IoU of 0.724, enabling actionable investigation timelines.
- **Red herring resilience —** deliberate exclusion of branch-level metrics, correlated features, and dataset-specific artefacts improves private-phase robustness.
- **Production readiness —** the modular pipeline (ingest → feature → model → window) maps directly to a real-time bank AML workflow.

This solution demonstrates that money mule detection at scale is achievable with a well-engineered feature set and a carefully regularised gradient boosting model — no deep learning or distributed compute required. The personalised window detection module adds direct operational value by telling investigators not just who is a mule, but precisely when the suspicious activity occurred.

Appendix: Submission File Format

The final submission CSV contains four columns:

Column	Type	Description
account_id	string	e.g. ACCT_000003
is_mule	float 0.0–1.0	LightGBM predicted probability
suspicious_start	datetime / null	Start timestamp of detected suspicious window
suspicious_end	datetime / null	End timestamp of detected suspicious window